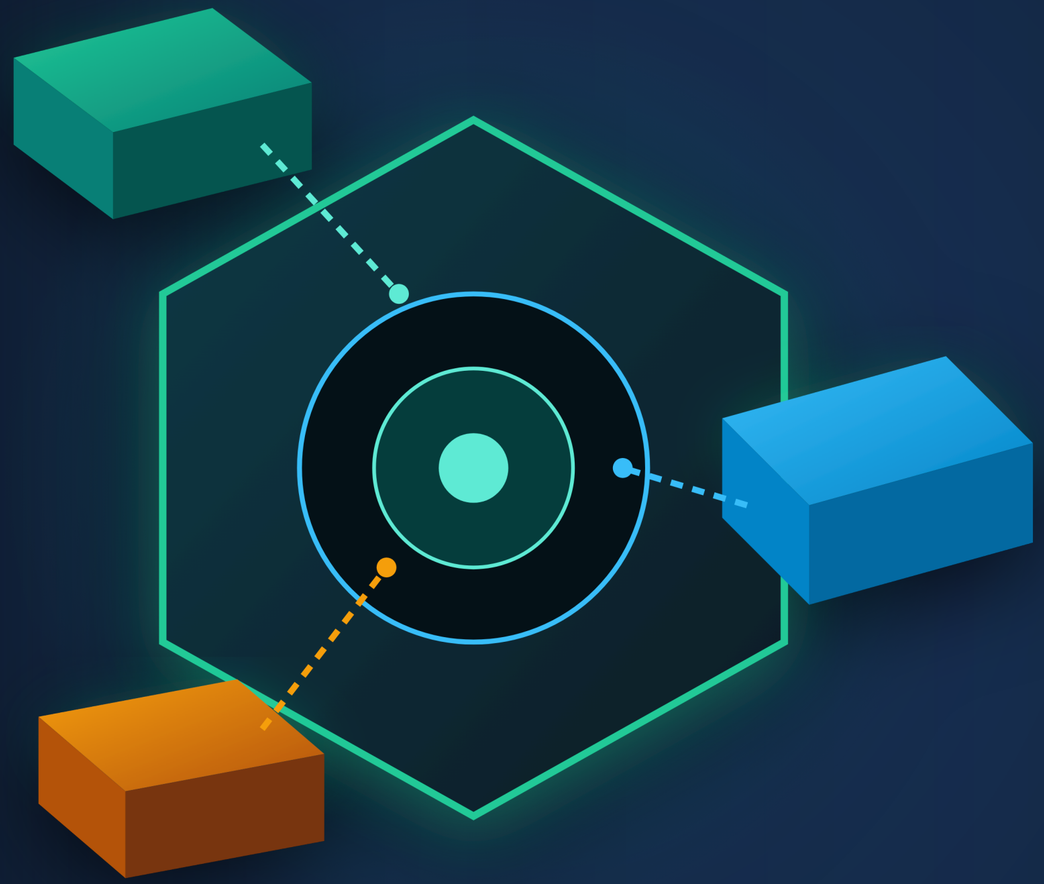
Software Engineering Lab **Lab 16**

Dependency Injection

Inversi Kontrol & Modularitas Kode



Lukman / lukman-ss

Konteks Arsitektur Hilang Tanpa Jejak

Hard-coded dependencies menghabiskan testability dan swap implementasi.

MASALAH NYATA

Kelas membuat objek dependensinya sendiri → tight coupling.
Ganti payment gateway memaksa ubah banyak tempat.
Test menjadi integration test yang lambat.

Ketika kelas menginstansiasi dependensinya sendiri, alasan teknisnya tidak terdokumentasi. Tim lama mengerti, tapi engineer baru bingung.

Tight coupling erases architectural context from code.

Mengapa Ini Penting

Coupling keras menghambat evolvability sistem jangka panjang.

DAMPAG

Test lambat karena bergantung pada status eksternal (API, database).

Perubahan vendor memicu modifikasi menyeluruh lapisan bisnis.
Arsitektur kaku karena tiap consumer terikat pada implementasi spesifik.

Separate construction from use for maintainable evolution.

Pemisahan Konfigurasi dari Penggunaan

Fase konfigurasi = buat objek; fase penggunaan = jalankan logika.

PRINSIP

Modul logika tidak boleh mencari atau membangun dependensinya.

Assembler/Composition Root yang bertanggung jawab merakit object graph.
Hollywood Principle: Don't call us, we'll call you.

External injection enables loose coupling.

DI vs Service Locator

Keduanya decorrelate, tapi mekanisme berbeda.

BEFORE: NAIVE / UNPROTECTED

CONSTRUCTOR INJECTION

- Dependensi muncul lewat parameter constructor.
- Tidak ada request eksplisit ke container.
- Jelas: Processor(PaymentGateway)

AFTER: OPTIMIZED / RESILIENT

SERVICE LOCATOR

✓ Consumer punya dependensi pada locator.

✓ Memanggil container.Get(...) secara aktif.

✓ Tersembunyi: processor hanya lihat Container

DI hides look-up; Service Locator exposes it explicitly.

Interface & Value Object

Money is VO; PaymentGateway is contract.

```
type Money struct {
    Amount   int
    Currency string
}
type PaymentGateway interface {
    Charge(m Money) error
}
```

CODE PATH

Interface enables swap; vo created directly without DI.

Constructor Injection

Processor menerima gateway lewat constructor.

```
type Processor struct {
    gateway PaymentGateway
}
func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}
func (p *Processor) ProcessPayment(amount int) error {
    if amount ⩽ 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m) }
```

CODE PATH

Constructor injection ensures valid state and explicit deps.

Skenario Kegagalan: Invalid Input

Validasi amount <= 0 mencegah panggilan sia-sia.

ProcessPayment(-10)

Result

Error: invalid amount, gateway NOT called

ProcessPayment(100)

Result

Money created, gateway.Charge() invoked

Input validation blocks infrastructure calls cleanly.

Bukti: Isolated Testing Tanpa Infrastruktur

MockGateway menggantikan RealGateway secara penuh.

```
mock := &MockGateway{}
proc := di.NewProcessor(mock)
err := proc.ProcessPayment(50)
if err ≠ nil { t.Fatalf("expected nil, got %v", err) }
if mock.ChargedMoney.Amount ≠ 50 { t.Errorf("wrong")
```

CODE PATH

Interface injection enables fully isolated unit tests.

Batasan Klaim

Tidak ada jaminan produksi otomatis.

PERNYATAAN YANG DAPAT DIBUKTIKAN

DI memudahkan testing tanpa infrastructure nyata.
MockGateway menggantikan network call secara penuh.
Tes berjalan dalam 0.00s per kasus.

"bukan berarti" behaving di production sama dengan test.

"tidak otomatis" performa atau defect rate menurun.

"perlu bukti tambahan" untuk skala enterprise.

Lab membuktikan testability; production butuh validasi terpisah.

Batas Integritas

Tes tidak membuktikan keamanan produksi.

DI LUAR CAKUPAN

Lifecycle management singleton/scoped/transient.

Performance overhead container vs direct new.
Pengurangan defect rate akibat DI.

Passing unit tests validate structure; production needs holistic validation.

Manual Wiring Tanpa Framework

YAGNI: Go stand-alone cukup membuktikan konsep.

ALUR KERJA

main.go menginisialisasi RealGateway.

NewProcessor(realGateway) – Constructor Injection.
Output: RealGateway charging 100 USD

Manual wiring bukan berarti tidak profesional; type-safe dan compile-time valid.

Payment Processor: Contrast

Processor vs BadProcessor pada kasus nyata.

CONTOH

Processor: NewProcessor(realGateway)

BadProcessor: NewBadProcessor(container)

Output keduanya identik: RealGateway charging ...

DI pattern yields better testability; Service Locator hides dependencies.

Review Kode: Anti-Pattern

BadProcessor menyembunyikan dependensi di container.

```
type BadProcessor struct {
    container Container
}
func (p *BadProcessor) ProcessPayment(amount int) error {
    return p.container.GetPaymentGateway().Charge(...)
}
```

CODE PATH

Service Locator membuat dependensi tak terlihat pada signature.

Checklist DI dalam Tim

Langkah implementasi yang direkomendasikan.

- ✓ Deklarasikan dependensi sebagai interface.
- ✓ Gunakan constructor injection sebagai default.
- ✓ Composition root SATU-satunya tempat buat implementasi konkret.
- ✓ Container.Get(...) di logic domain.
- ✓ Inject value object ke container.
- ✓ Constructor dengan banyak parameter (>12 sebagai sinyal).

Checklist memastikan DI implementation konsisten dan testable.

Kesalahan Umum

Yang sering dilakukan developer baru.

1. SERVICE LOCATOR SEBAGAI DI

Inject container ke kelas; panggil container.Get() di method.
Solusi: injeksikan interface spesifik yang dibutuhkan.

Masukkan Money, DateTime, Address ke container.

Solusi: buat langsung, tidak butuh interface.

Abstraction tanpa manfaat = overhead yang tidak perlu.

Biaya DI

Tidak ada Solution gratis; ada trade-off.

KEUNTUNGAN

Testing terisolasi tanpa infrastructure.
Swap implementasi lewat composition root.
Clear dependency contracts.

Konfigurasi ekstra di main.go.

Tracing kesulitan (behavior dipisah dari construction).

Refactoring penting untuk memilih DI framework.

DI improves testability; adds configuration overhead that must be managed.

Kesimpulan

DI meningkatkan modularitas, bukan penyelesaian semua masalah.

INSIGHT UTAMA

Constructor injection default untuk testability.
Avoid Service Locator di domain logic.

Mock injection = testing isolation tanpa network.
Value objects langsung dibuat, tidak lewat DI.

DI memisahkan konfigurasi dari penggunaan; bukan silver bullet.

8 Poin Kunci

Ringkasan prinsip Dependency Injection.

1. Sepetakan konfigurasi dari penggunaan
2. IoC umum; DI spesifik untuk dependensi
3. Constructor injection default
4. Service Locator dilarang di domain
5. Program to interfaces
6. Mock injection untuk pengujian
7. Value objects langsung dibuat
8. Over-injection = SRP violation

Eight core principles guide DI adoption in Go applications.

Komponen DI

Tabel komponen dan lokasinya.

Komponen	Lokasi	Peran
PaymentGateway	internal/di/gateway.go	Interface abstraksi payment provider
RealGateway	internal/di/gateway.go	Implementasi produksi
Money	internal/di/gateway.go	Value object
Processor	internal/di/processor.go	Service dengan Constructor Injection
Container	internal/di/locator.go	Interface Service Locator
BadProcessor	internal/di/locator.go	Service dengan Anti-pattern
MockGateway	tests/processor_test.go	Test double

Clear component mapping enables modular testing.

Cara Menduplikasi

Jalankan langkah ini di mesin lokal.

```
cd labs/16-dependency-injection
go test -v ./...    # 6 test cases, semua PASS
go test -race ./... # race-detector clean
go run ./cmd/demo   # jalankan demo DI vs Service Locator
```

CODE PATH

Reproduction requires only Go standard library, no external dependencies.

Hasil Verifikasi

Semua test lulus dengan race detector bersih.

6

Test Cases

3 untuk Processor • 3 untuk BadProcessor

0

Failures

Semua lolos

PASS

Race Detector

go test -race bersih

Comprehensive test coverage validates DI implementation behavior.

Sebelum vs Sesudah DI

Perbandingan pola yang direkomendasikan.

BEFORE: NAIVE / UNPROTECTED

TANPA DI

- Processor membuat gateway sendiri.
- Hard-coded pada RealGateway.
- Test lambat, butuh network.

AFTER: OPTIMIZED / RESILIENT

DENGAN DI

✓ Processor menerima interface.

✓ Swappable lewat constructor.

✓ Test cepat, mock-only.

DI mengekspos dependency lewat kontrak, bukan hard-code.

DI untuk Kode Lebih Bersih

DI enables modular, testable code but adds configuration overhead.

Software Engineering Lab 16 – Dependency Injection

internal/di/gateway.go • internal/di/processor.go • internal/di/locator.go
cmd/demo/main.go • tests/processor_test.go
README.md • content/*

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman-ss

Sumber: commit 8772235
Test passing: 6 test cases, 0 failures.
Race detector: clean (go test -race ./...)
Manual wiring demo; production may need framework.